Lab 5

Integrantes:
- Francis Aguilar,
- César López,
- Jose Marchena, 22398

Repo:
https://github.com/Czar272/Lab5VxC.git

In [1]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:
```
# SET DE CONTROL
img_left_control = cv2.imread("db/control_1.jpeg")
img_right_control = cv2.imread("db/control_2.jpeg")

# SET EXPERIMENTAL
img_left_exp = cv2.imread("db/experimental_left.jpeg")
img_right_exp = cv2.imread("db/experimental_right.jpeg")
```

In [3]:
```
def stitch_images(img_left, img_right, ratio_test=0.7, ransac_thresh=5.0):

    #Convertir a escala de grises
    gray_left = cv2.cvtColor(img_left, cv2.COLOR_BGR2GRAY)
    gray_right = cv2.cvtColor(img_right, cv2.COLOR_BGR2GRAY)

    #Detectar puntos clave con SIFT
    sift = cv2.SIFT_create()
    keypoints_left, descriptors_left = sift.detectAndCompute(gray_left, None)
    keypoints_right, descriptors_right = sift.detectAndCompute(gray_right, None)

    #Matching con FLANN
    FLANN_INDEX_KDTREE = 1
    index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
    search_params = dict(checks=50)
    flann = cv2.FlannBasedMatcher(index_params, search_params)

    matches = flann.knnMatch(descriptors_right, descriptors_left, k=2)

    #Lowe Ratio Test
    good_matches = []
    for m, n in matches:
```

```
        if m.distance < ratio_test * n.distance:
            good_matches.append(m)

    if len(good_matches) < 4:
        raise ValueError("No hay suficientes matches para calcular la homografia")

    #Calcular homografia
    src_pts = np.float32([keypoints_right[m.queryIdx].pt for m in
good_matches]).reshape(-1, 1, 2)
    dst_pts = np.float32([keypoints_left[m.trainIdx].pt for m in
good_matches]).reshape(-1, 1, 2)

    M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, ransac_thresh)

    #Crear canvas
    h_left, w_left = img_left.shape[:2]
    h_right, w_right = img_right.shape[:2]

    canvas_width = w_left + w_right
    canvas_height = max(h_left, h_right)

    panorama = cv2.warpPerspective(img_right, M, (canvas_width, canvas_height))
    panorama[0:h_left, 0:w_left] = img_left

    #Blending simple
    panorama_gray = cv2.cvtColor(panorama, cv2.COLOR_BGR2GRAY)
    _, mask_panorama = cv2.threshold(panorama_gray, 1, 255, cv2.THRESH_BINARY)
    mask_inv = cv2.bitwise_not(mask_panorama)

    left_on_canvas = np.zeros_like(panorama)
    left_on_canvas[0:h_left, 0:w_left] = img_left

    img_left_part = cv2.bitwise_and(left_on_canvas, left_on_canvas, mask=mask_inv)
    img_right_part = cv2.bitwise_and(panorama, panorama, mask=mask_panorama)

    result = cv2.add(img_left_part, img_right_part)

    #Recortar columnas vacias
    rows, cols = np.where(result[:, :, 0] != 0)
    if len(cols) > 0:
        max_col = np.max(cols)
        result = result[:, :max_col + 1]

    return result
```

In [4]:

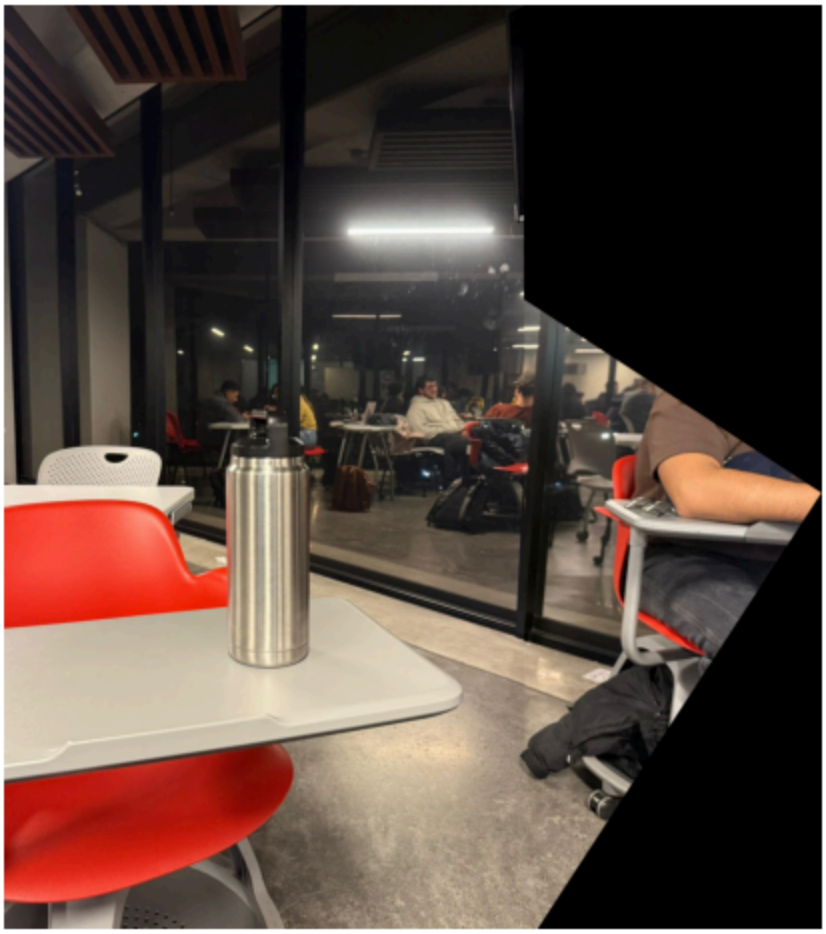
```
def show_image(img, title="Imagen"):
    plt.figure(figsize=(10,6))
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title(title)
    plt.axis("off")
    plt.show()
```

In [5]:
```
# Cargar imagenes del set control
img_left_control = cv2.imread("db/control_1.jpeg")
img_right_control = cv2.imread("db/control_2.jpeg")

# Ejecutar stitching
panorama_control = stitch_images(img_left_control, img_right_control)

# Mostrar resultado
show_image(panorama_control, "Panorama - Set Control")
```

Panorama - Set Control



In [6]:
```
# Cargar imagenes del set experimental
img_left_exp = cv2.imread("db/experimental_left.jpeg")
img_right_exp = cv2.imread("db/experimental_right.jpeg")

# Ejecutar stitching
panorama_exp = stitch_images(img_left_exp, img_right_exp)

# Mostrar resultado
show_image(panorama_exp, "Panorama - Set Experimental")
```

Panorama - Set Experimental

Task 2

Task 2.1

Solicite a su asistente de IA (ChatGPT/Claude/etc) que genere un script para resaltar estas discrepancias,
Se le deja este prompt como sugerencia para su uso inicial:

"Actúa como ingeniero de visión artificial. Tengo dos imágenes: la imagen base (izquierda) y la imagen warped (derecha transformada). Escribe un código en Python que genere una imagen compuesta (blending) donde la imagen base se muestre en el canal Rojo y la imagen warped en el canal Cian (Verde+Azul). Esto debe crear un efecto de anaglifo donde los píxeles perfectamente alineados se vean en escala de grises y los errores se vean como bordes de colores."

In [ ]:
```
def create_anaglyph(base_img_path, warped_img_path, output_path=None):
    # Leer imágenes
    base = cv2.imread(base_img_path)
    warped = cv2.imread(warped_img_path)

    if base is None or warped is None:
        raise ValueError("Error al cargar alguna de las imágenes.")

    # Asegurar mismo tamaño
    if base.shape != warped.shape:
        print("Redimensionando warped para que coincida con base...")
        warped = cv2.resize(warped, (base.shape[1], base.shape[0]))

    # Convertir a escala de grises si son RGB
    if len(base.shape) == 3:
        base_gray = cv2.cvtColor(base, cv2.COLOR_BGR2GRAY)
    else:
        base_gray = base.copy()

    if len(warped.shape) == 3:
        warped_gray = cv2.cvtColor(warped, cv2.COLOR_BGR2GRAY)
    else:
        warped_gray = warped.copy()

    # Crear imagen anaglifo
    anaglyph = np.zeros((base_gray.shape[0], base_gray.shape[1], 3), dtype=np.uint8)

    # Canal Rojo ← base
    anaglyph[:, :, 2] = base_gray

    # Canal Verde y Azul ← warped (Cian)
    anaglyph[:, :, 1] = warped_gray
    anaglyph[:, :, 0] = warped_gray

    # Guardar si se especifica ruta
    if output_path is not None:
        cv2.imwrite(output_path, anaglyph)

    return anaglyph
```

In [18]:
```
result = create_anaglyph("db/experimental_left.jpeg", "db/experimental_right.jpeg",
"anaglyph.png")

show_image(result, "Resultado")
```

Resultado

Task 2.1 – Visualización del Error

Código realizado en distancias.py

Objeto	Distancia estimada real	Coordenada X Fantasma 1 (Rojo)	Coordenada X Fantasma 2 (Cian)	Disparidad (x1-x2) (px)
A (Cercano)	400	320.9	757.3	436.4
B (Medio)	100	19.6	182.4	162.8
C (Fondo)	100	669.0	826.6	157.6

Task 3

Pregunta 1: Observe la columna de "Disparidad". ¿Existe una tendencia clara? ¿Cómo se comporta el valor de la disparidad a medida que aumenta la distancia real del objeto?

Si hay una tendencia, a mayor cercanía del objeto, mayor es la disparidad, y a mayor distancia, menor es la disparidad.

Pregunta 2: Si definimos Z como profundidad y d como disparidad. Basado en su abla, ¿la relación se parece más a una función lineal ($d = k \cdot Z$) o a una inversa ($d = k/Z$)? Grafique sus 3 puntos si es necesario para visualizarlo.

Se asemeja más a una función inversa, lo cual tiene sentido, cuando un objeto está muy cerca, pequeños cambios
en profundidad producen grandes cambios en disparidad. Cuando el objeto está
lejos, grandes cambios en profundidad producen cambios mínimos en disparidad.
Los objetos B y C tienen disparidades casi idénticas (162.8 vs 157.6) a pesar
de estar a distancias distintas, lo que confirma que la curva se "aplana"
rápidamente, comportamiento propio de una inversa.

Pregunta 3: Usted acaba de experimentar con los principios de la Estereoscopía. Explique por qué el algoritmo de Panoramas (Homografía) falló en alinear los objetos cercanos. ¿Qué información tridimensional estaba "escondida" en ese fallo?

Es porque se asume que la escena es plana y que todos los objetos están a la misma distancia. La Homografía puede alinear correctamente los objetos del fondo, pero los objetos cercanos se desplazan un ángulo diferente entre las dos vistas debido al paralaje. Ese desplazamiento diferencial no puede ser corregido por una sola Homografía para todo. La información "escondida" es precisamente la misma disparidad, ya que no es capaz de tomar en cuenta la profundidad del objeto

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.